SharePoint REST endpoint anatomy

The shape that mirrors CSOM, anchored at _api/ — and the four ways to address a list.

A URL stays a CSOM path

https://contoso.sharepoint.com/sites/hr/_api/web/lists/getbytitle('Projects')/items

web-relative	marker + scope	CSOM node	projection
resolved via pageContext.web	/_api/ selects the REST surface	web / lists / sites / users …	id / title / items / fields …

Four ways to address a list

By title · friendly

/_api/web/lists/getbytitle('Projects')/items

By id · stable

/_api/web/lists('a8d2…-…-…-…')/items

Cross-site · server-relative

/_api/sites/root/web/lists/getbytitle('Projects')/items

Root · tenants without host names

/_api/site/rootweb/lists/getbytitle('Projects')/items

SharePoint automatically resolves pageContext.web.absoluteUrl + '/_api/…' for you — but knowing what lands in each segment keeps surprises out of your code.

Three surfaces, one model

Same data surfaces three different ways — pick deliberately; do not all three at once.

1 SharePoint REST

/_api/ and /_api/v2.0/

- Default SPA + PowerShell surface
- Mirrors CSOM 1:1, type names shaped like SP.Data.ProjectsListItem
- Accepts odata=verbose or odata=nometadata
- v2.0 streamlines types; v3 preview, prefer v1
- Use this for 95% of cases

2 listData.svc

legacy Atom/XML listdata

- Predecessor, now deprecated
- Returns OData Atom or JSON
- Older ListData endpoints still work in some on-prem farms
- No support for modern types
- Migrate anything you find there

3 SOAP / ASMX

lists.asmx, copy.asmx ...

- Server-side era, kept for legacy
- XML envelopes, verbose
- Used by SharePoint Designer 2010
- Not callable from a browser or an SPFx bundle
- Do not start any new code here

The metadata endpoint

```
// discover types & properties
GET /_api/web/lists/getbytitle('Projects')/$metadata

for backward compatibility: odata=verbose
for client payloads: odata=nometadata
minimum metadata: odata=minimalmetadata
```

Decision in one line

REST is the daily driver — use it for everything new.

listData.svc still hangs around; treat it as legacy.

SOAP lives only on server code paths — we ignore it.

$metadata is your mirror when the type you need feels unknown.

CRUD verbs at a glance

Five verbs are enough. SharePoint REST is mostly POST with X-HTTP-Method in disguise.

GET

Read

No body. Read once, read many, query options. Safe to retry.

POST

Create

Body describes the new resource. Returns 201 + + payload. Needs digest.

MERGE

Partial update

SharePoint's cousin of PATCH. Omit fields you don't don't change. Needs digest + ETag.

PUT

Replace

Full replacement. Send every required field. Rare for Rare for items; useful for files.

DELETE

Remove

Files, items, lists, sites. Recycle bin catches most most deletes; second-stage too.

The trick: POST + X-HTTP-Method

Write verbs do not travel directly — you POST and advertise the verb you actually meant.

```
// partial update (send only changed fields)
POST /_api/web/lists/getbytitle('Projects')/items(12)
X-HTTP-Method: MERGE
If-Match: *

// delete the same way
POST /_api/web/lists/.../items(12)/recycle()
X-HTTP-Method: DELETE
```

Headers you must remember

Accept

application/json;odata=nometadata

Content-Type

application/json;odata=verbose

X-RequestDigest

digest value from contextinfo

If-Match

ETag or * for unconditional

Real writes: digest, ETag, headers

Three movements complete almost every write — fetch the digest, read the ETag, then update with both.

1 — Acquire a request digest

```
POST /_api/contextinfo
Content-Length: 0

// returns FormDigestValue
d.GetContextWebInformation
.FormDigestValue
```

2 — Capture the ETag

```
GET /_api/web/lists/getbytitle('Projects')/items(12)

// response header
ETag: "3"

// or use If-Match: * if you
// don't care who edited last
```

3 — Write with both keys

```
POST .../items(12)
X-HTTP-Method: MERGE
If-Match: "3"
X-RequestDigest: 0x...
```

Create a list item

```
POST /_api/web/lists/getbytitle('Projects')/items
Content-Type: application/json
X-RequestDigest: 0x6E1B...

{
"Title": "Apollo rollout",
"Status": "Active",
"DueDate": "2026-12-15T00:00:00Z"
}

// 201 Created, body has new id
```

Update guarded by If-Match

```
POST /_api/web/lists/.../items(12)
X-HTTP-Method: MERGE
If-Match: "3"

{
"Status": "Closed",
"PercentComplete": 100
}

// 204 No Content on success
// 412 Precondition Failed if stale
```

Delete (recycle)

```
POST /_api/web/lists/.../items(12)/recycle()
X-HTTP-Method: DELETE
If-Match: *

// moves to Recycle Bin
```

Want it gone forever? Add an admin-only second-stage on the recycle bin, recycle bin, never loop SendToRecycleBin in a flow.

$filter, $orderby, get more specific

Operators, ordering, and the index columns that decide whether your query is fast or fatal.

$filter operators

Operator	Meaning	Example
eq	equals	Status eq 'Active'
ne	not equal	Status ne 'Closed'
gt / ge / lt / le	range compare	Modified gt '2026-01-01'
startswith	prefix match	startswith(Title,'Apollo')
substringof	contains (note order)	substringof('roll',Title)
and / or / not	compose clauses	Status eq 'A' and Due lt today
$orderby	asc or desc	$orderby=Modified desc
$select	trim payload surface	$select=Id,Title,Status
$expand	navigate to related	$expand=Author,Editor

Composite in real life

```
GET /_api/web/lists/getbytitle('Projects')/items
?$select=Id,Title,Status,Modified
&$filter=Status eq 'Active'
&$orderby=Modified desc
&$top=50
&$expand=Editor/Title
```

Index what you filter on

A non-indexed column scans the whole list. Lists > 5 000 items start to hurt — hurt — add indices through the column settings or via PowerShell before before shipping.

Combine with $top to keep payloads modest, then $select ruthlessly.

Pagination: read safely to the last row

Three flavours of paging, all live at __next. Use server-driven wherever you can.

Skip and top

```
?$top=50 &$skip=100
```

Manual but brittle — rows shift if the list is edited, and you can still over-fetch. Best for tiny lookups.

__next loop

```
{'odata.nextLink': '/sites/...'}
```

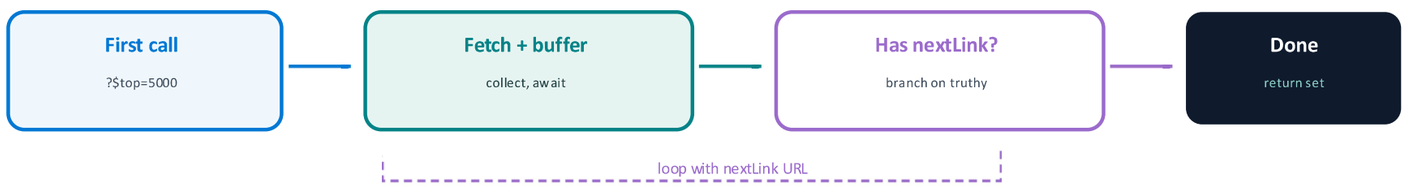
Follow the nextLink until it disappears. Resilient to edits, atomic per page. The page. The recommended default.

CAML for opaque lists

```
GetItems(query=...)
```

Reach for it only when OData can't express the query (joins, projections, scoped projections, scoped lookups, RollUp).

The server-driven paging loop



Pattern, not promise

```
let url = url0;
let items = [];
do {
const page = await get(url);
items.push(...page.value);
url = page["odata.nextLink"];
} while (url);

// batch-process pages with a
// concurrent bounded queue
```

Microsoft Graph fundamentals

One URL, many workloads. Forms a uniform API across Microsoft 365 — same shape as SharePoint REST, broader scope.

The single endpoint

```
https://graph.microsoft.com/v1.0/me/drive/root/children
```

Two namespaces: v1.0 is shipping, beta is forward-looking

- /v1.0/ — GA, stable, SLA-backed, what you ship on
- /beta/ — preview; properties and paths may break without warning
- Never mix a v1.0 token with the beta URL across a service boundary

SDKs that ship the typing

- JavaScript / TypeScript — @microsoft/microsoft-graph-client
- .NET — Microsoft.Graph (Kiota-generated)
- Python / Go / Java / PHP / Ruby — official client libs
- PnPjs wraps Graph when you want one fluent client for both surfaces

A typical Graph payload

```
// GET /v1.0/me
{ "id": "00aa…", "displayName": "Pat Rowan", "mail": "pat@contoso.com",
"jobTitle": "Developer", "@odata.context": "…" }
```

Conventions to internalise

@odata.context shape is documented in code
@odata.nextLink paging signal (similar to SP)
HTTP semantics match REST — 201 / 204 / 404
Errors always carry a JSON error envelope

Graph vs SharePoint REST

Same problems, different reach. Pick by what you must touch, not by which is newer.

Concern	SharePoint REST	Microsoft Graph
Endpoint	contoso.sharepoint.com/_api/	graph.microsoft.com/v1.0/
Scope	SharePoint only — lists, sites, files, users in SP context	Entire M365 — users, groups, mail, Teams, OneDrive, identity
Type names	SP.Data.ProjectsListItem etc.	driveItem, site, listItem, user ...
Auth	bearer + digest for writes	OAuth bearer — delegated or application
Throttling	per-resource; 429 + Retry-After	tenant-wide ~4 000 / 20s; delta helps
Niche features	taxonomy, navigation, web parts, feature flags	search, identity, meetings, chats, mail

Rule of thumb

If the work stays inside SharePoint, REST is fine. The moment you need identity, group lifecycle, mail or Teams, switch to Graph. PnPjs hides the choice while keeping it honest.

Two identities, one token bridge

Delegated flows ride the user. Application flows ride the app itself. Pick the right one — then pick the right endpoint.

Delegated · user-in-the-loop

Auth code flow + PKCE

- Browser redirects to Entra, user grants consent
- Token arrives as a bearer scoped to Graph + SharePoint
- Default for SPFx: AadTokenProvider, MSGraphClient
- Permissions: .All tenant-wide or .Selected opt-in
- Best for: user-scoped reads & writes the user should do

Application · no user

Client credentials flow

- No user; token is bound to a managed identity / app registration
- Use only in Azure Functions, Logic Apps, daemons
- Permissions carry .All and need Graph admin consent
- NEVER blend into a browser bundle — secrets leak
- Best for: jobs and migrations that must run unattended

Tokens in SPFx — you almost never mint one

```
// MSGraphClient brings the token with it
const graph = await this.context.msGraphClientFactory
.getClient('3'.repeat(1)); // v1.0 + .default

// Or fall back to AadTokenProvider (legacy path)
const tokenProvider = await this.context.aadTokenProviderFactory
.getTokenProvider();
const token = await tokenProvider.getToken('https://graph.microsoft.com');
```

What the SPFx context gives you

aadTokenProviderFactory
hands you a token you can pass to REST

msGraphClientFactory
stores the v1.0 client; auto-refreshes tokens

spHttpClient
injects the SharePoint bearer header automatically

Scopes & least-privilege

Ask for the smallest scope that fits. Read scopes first — write scopes second — .All only when you mean it.

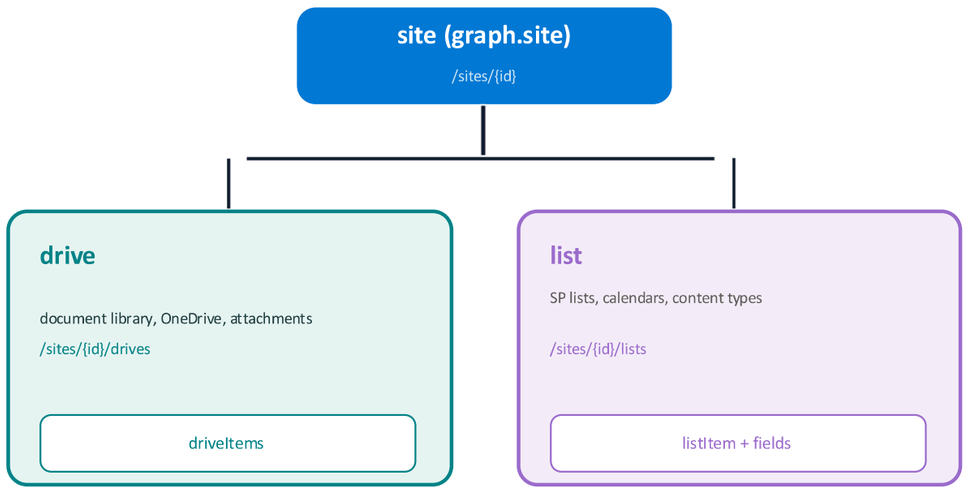
Scope	Type	What it allows	When to use
Sites.Read.All	delegated	Read sites, lists, items across all SPO sites	Default for read-only dashboards
Sites.ReadWrite.All	delegated	Read & write items across all sites	Form editing, item updates
Files.ReadWrite	delegated	User's own OneDrive + drive items	My-files scenarios
Group.Read.All	delegated	List M365 groups, read memberships	Team and group browsers
Sites.Selected	application	Touch only chosen sites via explicit grants	Recommended for new web parts
User.ReadBasic.All	delegated	Read basic profile for all users	Person picker, mentions, profile chips
Mail.Read	delegated	Read signed-in user's mail	Cross-M365 scenarios
.default	both	Use the statically-configured scopes for the app	SPFx default in web part manifest

Tip: scope acknowledgement happens in the sppkg manifest (web part properties) and again at grant time. Admin consents for Sites.Selected; user consents for read-only delegated.

SharePoint entities, Graph-shaped

Each SharePoint concept has a Graph path. Same data, mirrored endpoint tree.

Resource tree, mirrored



Permissions live on the resource that owns them

driveItem, listItem, site, list, drive each carry their own /permissions navigation. Obtain a shareable link, an invite grant, or read existing roles — always scoped to the resource.

Real endpoints

find a site
/v1.0/sites/contoso.sharepoint.com:/sites/hr

list the lists
/sites/{id}/lists?$expand=columns

items + fields
/sites/{id}/lists/{id}/items?$expand=fields

all drives & items
/sites/{id}/drives | /drives/{id}/root/children

permissions
/sites/{id}/permissions | .../items/{id}/permissions

Graph returns drives and lists as siblings of the site — they are navigations, not separate roots. Fetch the site once, walk the rest.

Working with Users

One endpoint, four jobs — load /me, look up by UPN or id, page delta, walk transitive membership.

A few canonical calls

```
// the signed-in user
GET /v1.0/me

// look up by UPN or by id
GET /v1.0/users/pat@contoso.com?$select=id,displayName,jobTitle

// find users whose name starts with P
GET /v1.0/users?$filter=startswith(displayName,'P')
&$select=id,displayName,mail,userPrincipalName
&$top=25

// transitive membership walk
GET /v1.0/groups/{id}/transitiveMembers

// sync directory changes
GET /v1.0/users/delta
```

Identify by id, not mail

Mail changes; immutable ids don't. Store user ids in your data model — resolve display strings per request.

Use $select, always

Unscoped /users returns dozens of properties you do not need. Trim cache the result.

Delta for sync, not browse

/users/delta returns additions, updates and tombstone removals. Best for export jobs; not for search UIs.

Industry scenario — HR onboarding pull

A new-hire hub in HR contains a column for the manager's UPN. Resolve once with ?$select=id,displayName,jobTitle,department, cache by id in the SPFx state, prefer ids for downstream writes.

Groups & the SPO site it backs

M365 groups and security groups look similar; only one becomes a SharePoint site.

Microsoft 365 group

- Carries an SPO site, mailbox, Teams channel, Planner, OneNote notebook
- ResourceProvisioningOptions include Team
- Created via POST /groups; manage members through /groups/{id}/members / owners
- Visibility: Private, Public, HiddenMembership
- Best for: collaboration hubs, projects, departments

Security group

- No site, no Teams, no mailbox — membership only
- Used inside Entra ID role assignments, app permissions
- Created via POST /groups with mailEnabled=false,securityEnabled=true
- Imported via AD syncs, dynamic rules in Entra
- Best for: assignment, classification, sharing

Lifecycle of an M365 group — one identity, many surfaces



Deleting the M365 group archives the email, removes the Teams channel, and recycles the backing site. Walk this tree with /groups/{id} — one call returns the team, site, drive, plans and conversations.

Sites & lists via Graph

Look up a site once, then walk its lists. Read fields inline — don't round-trip after discovery.

From discovery to reads to follows

```
// 1 · resolve a site by hostname + path
GET /v1.0/sites/contoso.sharepoint.com:/sites/hr

// 2 · list all lists with their columns in one call
GET /sites/{id}/lists?$expand=columns($select=name,displayName)

// 3 · read items, inline fields
GET /sites/{id}/lists/{id}/items
?$expand=fields($select=Title,Status,Modified)

// 4 · follow the site for the current user
POST /me/followedSites/add
{ "value": [{ "id": "<siteId>" }] }

// 5 · create a list (synchronous, simple)
POST /sites/{id}/lists
{ "displayName": "Tickets", "list": { "template": "genericList" } }
```

Address sites by URL

Encode `host:/sites/path`; Graph parses the URL and resolves to site id. No need to know the guid up front.

$expand makes fields real

Without it the items come back as `{"id":"..."}` only. With it you get the column values you actually want.

Followed-sites is SP-native

The `followedSites` collection backs SharePoint's news feed. Adding to it surfaces the site to the user immediately.

Tip. Use `/search/query` when you don't know which list the user means. Field filtering and Kairos-style hits live at the Graph search vertical — tall one in module 5.

Files, drives, driveItems

A drive is a file container; a driveItem is a node — file or folder — inside it. Same tree shape as OneDrive.

A drive in four shapes

document library · /sites/{id}/drives
user's OneDrive · /users/{id}/drive
group drive · /groups/{id}/drive
site default drive · /sites/{id}/drive

Walk the tree

/drives/{id}/root
/drives/{id}/root/children
/drives/{id}/items/{id}/children
/drives/{id}/items/{id}/versions
/drives/{id}/items/{id}/permissions

driveItems are typed

file has file facet & mime
folder has folder facet & childCount
size,webUrl,lastModified common to both
cTag changes when content or metadata change

Each item carries the id used in every subsequent op — copy, move, perm, version.

Sample payload


```
{
"id": "01XYZ...",
"name": "Brief Q3.docx",
"size": 14592,
"webUrl": "https://...",
"lastModifiedDateTime":
"2026-09-12T08:14Z",
"file": { "mimeType": "application/..." }
}
```


driveItems vs listItems

driveItem documents, files, attachments — the natural surface

listItem structured rows with fields, taxonomy, content types

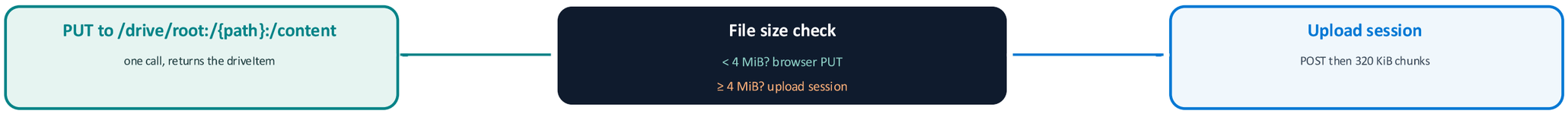
Document libraries expose both; pick by intent, not by tradition.

// id never expires — webUrl may change when sites are renamed. Store ids.

Uploads, conversions, views

Pick the upload path by size; render previews before the user opens; convert only when needed.

Upload path decision



Sequential chunks of up to 60×320 KiB, last partial chunk OK. Graph returns an *uploadUrl*; commit on the last chunk; cancel with DELETE on the same URL.

Small file · one PUT

```
PUT /drives/{id}/root:/Briefs/Q3.docx:/content
Content-Type: application/octet-stream

// bytes < 4 MiB; file replaces existing path
// 200 Created / 200 OK returns id, webUrl, eTag

use this for icons, attachments, profile photos
```

Large file · chunked

```
POST /drives/{id}/items/{parentId}:/{name}:/createUploadSession
{ "item": { "@microsoft.graph.conflictBehavior": "replace" } }

// PUT 320 KiB chunks to uploadUrl,
// range headers dictate offset,
// final commit when done
```

/drives/{id}/items/{id}/content?format=pdf · /thumbnails/0/medium

/items/{id}/preview · /items/{id}/permissions

PnPjs v4 namespaces

A fluent TypeScript wrapper with first-party typings — same data, less boilerplate. Default for new SPFx.

@pnp/sp

- Wraps SharePoint REST
- Models the CSOM tree: web.lists.getByTitle(...)
- Knows taxonomy, navigation, web parts, security
- Best for SP-specific nuance

@pnp/graph

- Wraps Microsoft Graph
- Same fluent shape: graph.me.drive etc.
- Beta version carried on the side
- Best for cross-M365 reads/writes

@pnp/core

- Runtime plumbing
- Behaviours, caching, batching, parsers, telemetry
- spfi() · graphfi()
- Always imported first

Wiring in SPFx

```
// 1 · on every shared service file
import { spfi, SPFx } from "@pnp/sp";
import { graphfi, MSGraph } from "@pnp/graph";

// 2 · per-instance, in the web part
exportconst sp = spfi().using(SPFx(ctx));
exportconst graph = graphfi().using(MSGraph(ctx));

// 3 · never re-create the sp / graph per render
```

Why over raw fetch

- TS types generated and versioned
- Batch and cache built in
- Boring auth wired once
- Active PnP community
- One client, two surfaces

CRUD and batch

Read once, refine. Write mundane verbs through chainable methods; queue many calls into a single roundtrip.

SharePoint CRUD with @pnp/sp

```
// CREATE
const created = await sp.web.lists.getByTitle("Projects").items.add({
Title: "Apollo rollout", Status: "Active", DueDate: new Date("2026-12-15")
});

// READ with select + filter + top
const items = await sp.web.lists.getByTitle("Projects").items
.select("Id,Title,Status,Modified")
.filter(`Status eq 'Active'`)();

// UPDATE partial, by id
await sp.web.lists.getByTitle("Projects").items.getById(12).update({
Status: "Closed", PercentComplete: 100
});

// DELETE (to recycle)
await sp.web.lists.getByTitle("Projects").items.getById(12).recycle();
```

Batch many into one

- Queue 20 ops — one batch per call
- Each op can succeed or fail independently
- Reduces latency, lowers throttle risk
- Default write-pattern for migrations

sample batch

```
await sp.createBatch();
list.items.getById(1).inBatch(b).update({...});
list.items.getById(2).inBatch(b).update({...});
list.items.getById(3).inBatch(b).recycle();
await b.execute();
```

Behaviours that earn their keep

DefaultInit, DefaultCachingWith, DefaultQueryProps, ResolveCopySrc: chain them onto spfi() so every consumer picks themup for free.

Errors, retries, security

Assume failure — treat every status code as data, retry sensibly, ask for the smallest credential.

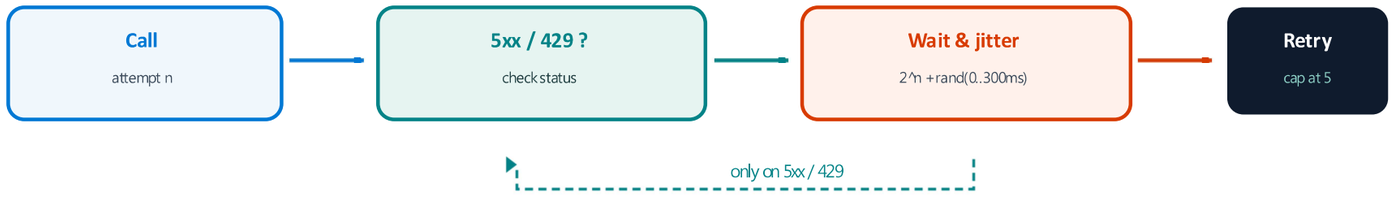
Graph error body shape

```
// 4xx / 5xx with body
{
 "error": {
 "code": "tooManyRequests",
 "message": "Retry after 30s",
 "innerError": { "code": "rateLimitExceeded" }
 }
}
```

Status, meaning, what to do

- 200 / 204 · success; 204 has empty body
- 400 · bad body — validate before send
- 401 · expired token — refresh, retry once
- 403 · forbidden — consent or scope missing
- 404 · gone — delete UI, do not retry
- 412 / 429 / 503 · contention — back off, honour Retry-After

Backoff with jitter



Cap retries; abort the call if Retry-After keeps growing; surface 403 / 404 to UI as actionable errors instead of retrying forever.

Security checklist

- ✓ Prefer Sites.Selected over .All
- ✓ Validate ETags on writes; avoid If-Match: *
- ✓ Treat the digest like a credential
- ✓ Apply Conditional Access policies
- ✓ Mind CSRF on classic endpoints
- ✓ Log the request id, not user data

Lab: PnPjs + Graph in SPFx

One web part, two surfaces: SP list via PnPjs, Graph /me and /sites for context.

01 Scaffold

yo @microsoft/sharepoint
--framework react --isolated

Out:

- package.json with web part manifest
- src/webparts/*name*/<webPart>.ts
- tied Node 22 LTS, SPFx 1.21 baseline

02 Install & wire PnPjs

npm i @pnp/sp @pnp/graph
@microsoft/microsoft-graph-client

Out:

- define *sp* and *graph* via spfi()
- a single shared *api/* folder for services

03 Read list via PnPjs

sp.web.lists.getByTitle(...)
.items.select(...).top(50)();

Out:

- first 50 active rows in state
- busy & error slots wired to UI

04 Add Graph context

const client = await ctx
.msGraphClientFactory.getClient();

Out:

- *me.displayName*, *me.jobTitle* in header
- site meta via *../sites/root*

05 Wrap errors

safe(promise, ctx) returns
{data | null, error | null}

Out:

- 429 surfaced as retryable
- cancelled flag honoured, no React storms

06 Ship it

gulp bundle --ship
&& gulp package-solution --ship

Out:

- *.sppkg* ready for the App Catalog
- admin approves Graph scopes once

Acceptance criteria

Owner sees their name + a list of 50 active rows, unmount cancels in-flight calls, console is clean, sppkg deploys and the web part appears under the toolbox.